Box2D v2.0.2 Руководство.

Оригинал(англ.) : http://www.box2d.org/manual.html
Перевод: kkray для форума GameDev.ru

Erin Catto
Copyright © 2007-2008 Erin Catto

Содержание

---

1. Введение
    - 1.1. О технологии
    - 1.2. Необходимые условия
    - 1.3. Основные понятия
2. Привет Box2D
    - 2.1. Создание мира
    - 2.2. Создание GroundBox'a (бокса земли)
    - 2.3. Создание Динамичного Тела (Dynamic Body)
    - 2.4. Моделирование физического мира (Box2D)
    - 2.5. Сбор мусора
    - 2.6. Тестовый стенд
3. Интерфейс Box2D
    - 3.1. Управление динамической памятью
    - 3.2. Фабрики и определения
    - 3.3. Единицы измерения
    - 3.4. Пользовательские данные
    - 3.5. Использование C++
    - 3.6. Недовольным
4. Мир
    - 4.1. О мире
    - 4.2. Создание и уничтожение мира Box2D
    - 4.3. Использование мира
        - 4.3.1. Моделирование
        - 4.3.2. Просмотр объектов мира
        - 4.3.3. AABB запросы
5. Тела
    - 5.1. О телах
    - 5.2. Определение тела
        - 5.2.1. Масса
        - 5.2.2. Позиция и угол
        - 5.2.3. Торможение
        - 5.2.4. Параметры сна
        - 5.2.5. Снаряды
    - 5.3. Фабрика тел
    - 5.4. Использование тел
        - 5.4.1. Данные о массе
        - 5.4.2. Информация о состоянии тела
        - 5.4.3. Положение и скорость
        - 5.4.4. Силы и импульсы
        - 5.4.5. Преобразование координат
        - 5.4.6. Списки
6. Фигуры

6.1. О фигурах
6.2. Определение фигуры
6.2.1. Коэффициенты трения и восстановления
6.2.2. Плотность
6.2.3. Фильтрация столкновений
6.2.4. Сенсоры
6.2.5. Определение Окружности
6.2.6. Определение Полигона
6.3. Фабрика фигур
6.4. Использование фигур
7. Соединения
7.1. О соединениях
7.2. Определение соединения
7.2.1. Соединение "жесткий отрезок"
7.2.2. Болтовое соединение
7.2.3. Призматическое соединение
7.2.4. Талевое соединение
7.2.5. Передаточное соединение
7.2.6. Соединение с курсором мыши
7.3. Фабрика соединений
7.4. Использование соединений
7.4.1. Использование соединений "жесткий отрезок"
7.4.2. Использование болтовых соединений
7.4.3. Использование призматических соединений
7.4.4. Использование талевых соединений
7.4.5. Использование передаточных соединений
7.4.6. Использование соединения с курсором мыши
8. Контакты
8.1. О контактах
8.2. Обработчик контактов
8.3. Фильтрация контактов
9. Напоследок
9.1. Границы мира
9.2. Неявное уничтожение
10. Настройки
10.1. О настройках
10.2. Отклонения
10.3. Выделение памяти
11. Отладочная отрисовка

Глава 1. Введение

Содержание

1.1. О технологии 1.2. Необходимые условия 1.3. Основные понятия

1.1. **О технологии**

Box2D это библиотека для моделирования поведения твердых тел в двумерном пространстве. С ее помощью программист может заставить объекты двигаться реалистично с точки зрения физики, а мир выглядеть более интерактивным. В тоже время с точки зрения устройства игры, физический движок это лишь система процедурной анимации. Вместо того, чтобы платить аниматору или умолять его научить ваши объекты двигаться реалистично доверьте всю работу сэру Исааку Ньютону и трем его физическим законам!

Box2D написан на портируемом C++. А имена большинства типов в движке во избежание конфликтов имен начинаются с префикса "b2", надеемся, что этого будет достаточно.

1.2. Необходимые условия

В этом руководстве предполагается, что вы знакомы с основными физическими понятиями, такими как масса, сила, вращающий момент и импульсы. Если это не так, то пожалуй вам стоит сначала ознакомиться с уроками Криса Хэккера (Chris Hecker) и Дэвида Бэраффа (David Baraff). Вам совсем не требуется понимать их уроки в мельчайших деталях, но у них прекрасно получилось разъяснить основные концепции физики, знание которых поможет вам в использовании Box2D.

Википедия (http://wikipedia.org) также является отличным источником знаний по физике и математике. В некоторых случаях она бывает более полезна чем Гугл (http://google.com), так как содержит тщательно подобранную информацию.
Это совсем не обязательно, но если вы действительно хотите разобраться, как Box2D работает "внутри", вам следует взглянуть на эти статьи:
http://www.gphysics.com/downloads .

Box2D написан на C++, поэтому предполагается, что вы имеете имеет опыт в программировании на C++. Не следует выбирать Box2D в качестве первого проекта для программирования на C++.
Компилирование, линковка и отладка приложения не должны представлять для вас сложности.

1.3. Основные понятия

Box2D работает с несколькими базовыми объектами. В этой главе мы лишь кратко опишем эти объекты, а более подробные описания приведем позже.
Твердое тело (rigid body)
Предмет, состоящий из материала настолько прочного, что расстояние между любыми двумя точками этого предмета является постоянным при любых условиях. Он прочен как алмаз. В дальнейшем для удобства мы будем использовать слово *тело* вместо «твердое тело».
Фигура (shape)
Плоская геометрическая фигура, жестко прикрепленная к телу, применяется для расчетов столкновений. Фигуры имеют свойства присущие материалам: коэффициент

трения и упругость.
Ограничение (constraint)
Физическое соединение, ограничивающее свободу перемещение и/или вращения тела. На плоскости тело имеет 3 степени свободы (движение по вертикали, движение по горизонтали, вращение). Если мы возьмем тело и прибьем его гвоздем к стене (как маятник) то мы ограничим свободу тела стеной. В данном случае тело сможет только вращаться вокруг гвоздя, т.е. мы лишили тело 2-х степеней свободы.
Контактное ограничение (contact constraint)
Специальное ограничение, предназначенное для предотвращения проникновения тел друг в друга, а также моделирования трения и упругости тел. Вам не никогда не придется создавать такие ограничения, т.к. они создаются Box2D автоматически.
Соединение (joint)
Это специальное ограничение, используемое для соединения двух и более тел. Box2D поддерживает следующие типы соединений: "шарнирное", "призматическое", "жесткий отрезок" и другие. Соединения могут играть роль ограничителей (limits) или мускулов (motors). **Ограниченное соединение (joint limit)**
Как следует из названия, диапазон движений такого соединения ограничен. В качестве примера можно привести локоть человека.
Соединение-мускул (joint motor)
Приводит в движение все подсоединенные к нему тела в соответствии со степенями свободы их соединений. Например, можно использовать мускул, чтобы заставить локоть сгибаться и разгибаться.
Мир (world)
Мир объединяет все тела, фигуры и ограничения, взаимодействующие друг с другом. Box2D поддерживает создание нескольких миров, но обычно это не требуется.

Глава 2. Привет Box2D

Содержание
2.1. Создание мира 2.2. Создание GroundBox'a (бокса Земли). 2.3. Создание Динамичного Тела (Dynamic Body) 2.4. Моделирование физического мира (Box2D) 2.5. Сбор мусора 2.6. Тестовый стенд

Дистрибутив Box2D содержит проект *Hello World*. Программа создает огромный бокс земли и маленький динамичный бокс. Ее код не содержит никакой графики, вся информация выводится в виде текста.

2.1. **Создание Мира**

Каждая программа с использованием движка Box2D начинается с создания объекта мира. Этот объект — центр управления памятью, объектами и процессами моделирования.

Для его создания мы должны сначала определить ограничивающий бокс. Ограничивающие боксы используются в Box2D для ускорения расчетов столкновений тел. Размер бокса не критичен, но хороший его подбор увеличит производительность, впрочем, в случае выбора размера бокса применимо правило "лучше больше, чем

меньше".

```
b2AABB worldAABB;
worldAABB.lowerBound.Set(-100.0f, -100.0f);
worldAABB.upperBound.Set(100.0f, 100.0f);
```

> ***Внимание***
> Размер AABB мира всегда должен быть больше, чем регион где располагаются ваши "тела". Еще раз напомним, лучше сделать AABB очень большим, чем слишком маленьким. Если тело (возможно в процессе моделирования) пересекает границу AABB мира, то оно замораживается и исключается из процесса моделирования.

Теперь определяем вектор гравитации. Да, кстати, вы может направить вектор гравитации в сторону, а не вниз (или можете просто повернуть свой монитор :)). Также мы сообщаем миру, что телам позволено "засыпать" когда они находятся в состоянии покоя. Спящий объект не нуждается ни в каком моделировании поведения.

```
b2Vec2 gravity(0.0f, -10.0f);
bool doSleep = true;
```

Теперь создадим сам объект. В обычных условиях вам следует создавать объект мира на хипе (в динамической памяти) и сохранять указатель в одной из структур вашей игры. Однако и создание мира на стеке (в статической памяти) в этом примере прекрасно работает.

```
b2World world(worldAABB, gravity, doSleep);
```

Ну что же, теперь у нас есть мир (мир физики!), давайте начнем добавлять в него всякий хлам :)

2.2. **Создание** GroundBox'a (**бокса Земли).**

Для создания тела (body) нужно выполнить следующие шаги:

1. Сформировать определение тела: позиция, амортизация и т.д.
2. Создать тело при помощи объекта мира
3. Сформировать определение фигур: геометрия, трение, плотность и т.д.
4. Создать фигуры на теле.
5. Опционально, установить массу соответствующую прикрепленным к телу фигурам.

На шаге 1 мы создаем бокс земли. Для этого нам необходимо определение тела. В нем мы задаем начальную позицию тела.

```
b2BodyDef groundBodyDef;
groundBodyDef.position.Set(0.0f, -10.0f);
```

На шаге 2 определение тела передается в объект мира для создания самого тела. Объект мира не хранит ссылку на определение тела. Бокс земли создается как статичное тело.

Статичное тело не сталкивается с другими статичными телами и не способно двигаться. Box2D определяет, что тело статичное если его масса равна нулю. Тела имеют нулевую массу по умолчанию, следовательно, они по умолчанию статичные.

```
b2Body* ground = world.CreateBody(&groundBodyDef);
```

На шаге 3 мы создаем определение полигона (многоугольника) земли. Для этого мы используем упрощенную функцию SetAsBox, чтобы сформировать полигон земли как бокс, центр которого находится в начале координат родительского тела.

```
b2PolygonDef groundShapeDef;
groundShapeDef.SetAsBox(50.0f, 10.0f);
```

Функция SetAsBox принимает полуширину и полувысоту объекта. Так, в данном случае бокс земли имеет 100 единиц в ширину (ось X) и 20 единиц в высоту (ось Y). Box2d настроен для метров, килограмм и секунд. Таким образом, вы можете использовать размеры в метрах. Тем не менее, систему мер можно изменить, это тема обсуждается далее в этом документе.
В шаге 4 мы заканчиваем создание тела земли созданием на нем полигона.

```
groundBody->CreateShape(&groundShapeDef);
```

И снова напоминаем, Box2D не сохраняет ссылок на определения фигур и/или тел. Вместо этого он копирует данные внутрь структурыb2Body.
Заметьте, что каждая фигура обязана иметь родительское тело (тело к которому она присоединена) даже если эта фигура статичная. Однако, вы можете присоединить все статичные фигуры к одному телу. Это было сделано для статичных тел для того, чтобы сделать код внутри Box2D единообразным, универсальным, уменьшив таким образом количество потенциальных ошибок.

Здесь вы можете увидеть общую структуру построение кода. Имена большей части типов в Box2D начинаются с префикса b2. Это сделано, чтобы уменьшить вероятность конфликтов имен с вашим кодом.

2.3. **Создание Динамичного Тела (**Dynamic Body)

Итак, у нас уже есть тело "земли". Для создания динамичного тела используется такая же техника. Основное отличие, кроме размеров, состоит в том, что мы должны установить для него множество параметров присущих динамичным объектам.

Сначала создадим тело, используя `CreateBody`:

```
b2BodyDef bodyDef;
bodyDef.position.Set(0.0f, 4.0f);
b2Body* body = world.CreateBody(&bodyDef);
```

Затем создаём фигуру-полигон и прикрепляем ее к телу. Заметьте, что плотность (density) тела установлена равной 1. По умолчанию она равна нулю. Коэффициент трения (friction) поставим 0.3. После присоединения фигуры вызовем метод `SetMassFromShapes`, чтобы пересчитать массу тела на основе данных о присоединенных

фигурах. Это такой тонкий намек на то, что вы можете присоединять к телу (body) больше чем одну фигуру (shape). Если масса тела после пересчета окажется нулевой, то тело становится статичным. Заметьте, что тела имеют нулевую массу по-умолчанию, именно поэтому мы не вызывали `SetMassFromShapes`, когда создавали тело "земли".

```
b2PolygonDef shapeDef;
shapeDef.SetAsBox(1.0f, 1.0f);
shapeDef.density = 1.0f;
shapeDef.friction = 0.3f;
body->CreateShape(&shapeDef);
body->SetMassFromShapes();
```

Ну вот, с инициализацией закончили. Теперь можно начинать процесс моделирования.

2.4. Моделирование физического мира (Box2D)

Теперь у нас есть "земля" и динамичное тело, давайте же заставим законы Ньютона делать их работу. Но сначала рассмотрим пару новых понятий.

Для моделирования движения тел Box2D использует численное дифференцирование (а точнее метод Эйлера). Выгляди это так: мир Box2D содержит функцию Step(float timeStep,int iterations), имеющую два аргумента: время, которое необходимо смоделировать в мире и количество итераций для разрешения взаимодействий между объектами.

Обычно, физические движки для игр используют timeStep не более 1/60 секунды (т.е. "частота кадров" расчета физики не менее 60Гц), такого интервала вполне достаточно для реалистичного моделирования физики в играх. Вы можете задать другой timeStep, но в таком случае вам придётся более осторожно настраивать параметры вашего мира. Поэтому мы не советуем вам сильно изменять timeStep. Но очень нежелательно связывать временной шаг с частотой смены кадров графической подсистемы (исключая случаи, когда это действительно необходимо). Задаем timeStep:

```
float32 timeStep = 1.0f / 60.0f;
```

В коде Box2D большую часть занимает constraint solver ("решальщик" ограничений). Задача Constraint solver -- просчёт ограничений для правильного поведения тел при столкновениях. При просчёте одного ограничения не возникает особых трудностей. Однако при просчете нескольких ограничений, разрешение одного ограничения слегка нарушает остальные. Поэтому для получения хорошего результата необходимо произвести просчет всех ограничений несколько раз (т.е. сделать несколько итераций). В Box2d мы предлагаем использовать 10 итераций. Вы можете задать количество итераций по своему усмотрению, но необходимо помнить следующее правило «чем больше количество итераций, тем больше точность и меньше скорость просчёта ограничений» и наоборот «чем меньше количество итераций, тем меньше точность и больше скорость». Задаем количество итераций:

```
int32 iterations = 10;
```

Заметьте, что timeStep и количество итераций никак не связаны между собой. Итерация это НЕ под-шаг! Одна итерация это один расчет всех ограничений в каждый момент

времени. Мы можем использовать несколько проходов в каждый момент времени для увеличения точности.

Теперь мы готовы начать цикл моделирования. В вашей игре цикл моделирования может быть объединён с игровым циклом. В таком случае на каждой итерации игрового цикла необходимо вызывать b2World::Step. Обычно достаточно одного вызова, но это зависит от частоты кадров и величины timeStep.

Так как программа Hello World разработана в максимально упрощенном виде, она не имеет графической части. Чтобы не быть слишком скучным код выводит позицию и угол поворота динамичного тела. Ура! У нас есть цикл, который моделирует поведение динамичного тела на протяжении 60 временных отрезков, суммарное время моделирования равно 1 секунде.

```
for (int32 i = 0; i < 60; ++i)
{
      world.Step(timeStep, iterations);
      b2Vec2 position = body->GetPosition();
      float32 angle = body->GetAngle();
      printf("%4.2f %4.2f %4.2f\n", position.x, position.y, angle);
}
```

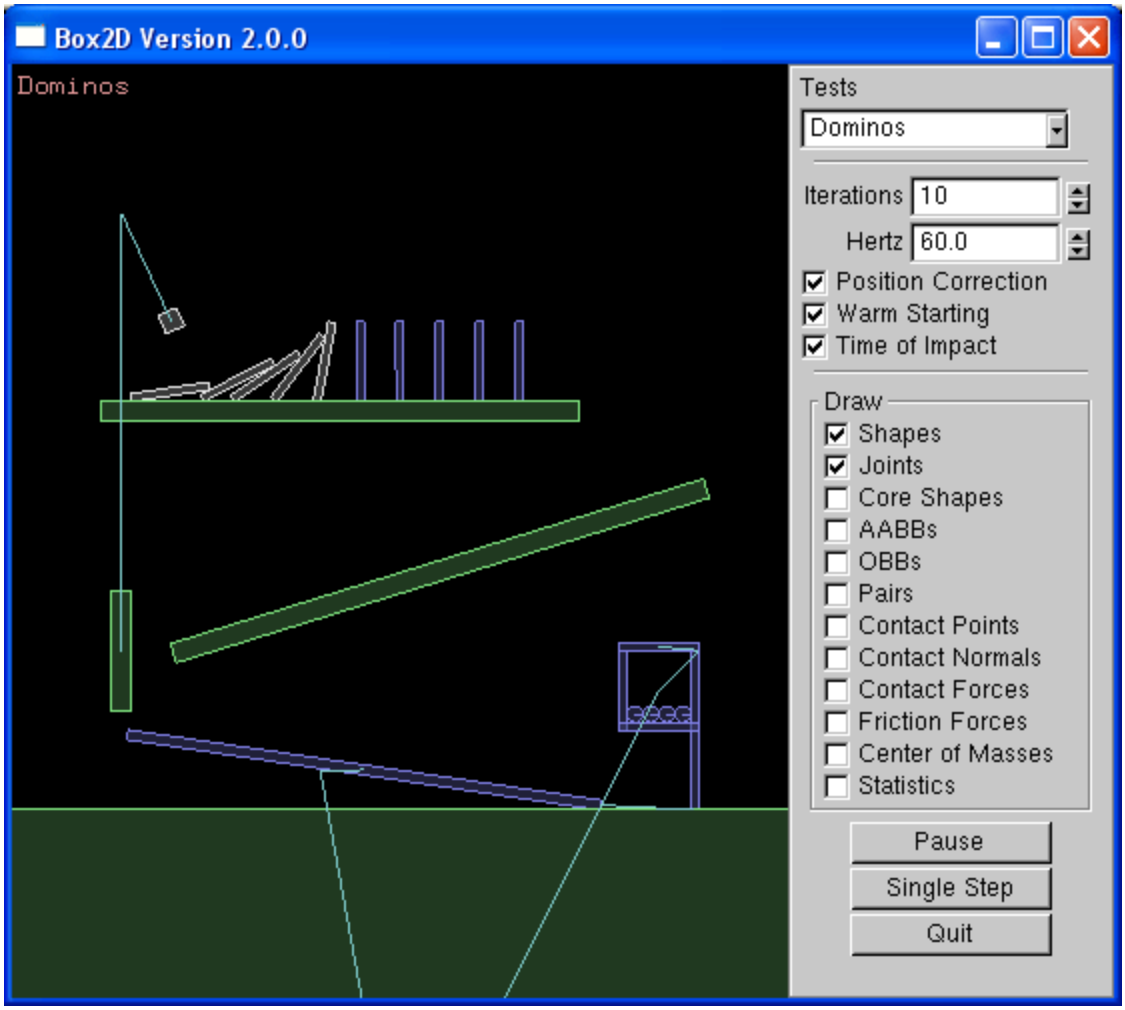
2.5. Сбор мусора

Когда объект мира покидает область видимости или уничтожается вследствие удаления указателя на него, вся память зарезервированная миром для тел и соединений между ними освобождается. Таким образом, вы будете должны обнулить указатели на все тела, фигуры и соединения которые хранятся в вашей игре, т.к. эти указатели станут недействительными.

2.6. Тестовый стенд

Когда пример `HelloWorld` вам покорится, посмотрите на испытательный стенд Box2D. Стенд представляет собой каркас для тестирования механизмов и построений. Вот некоторые из его особенностей:

- Движущаяся по вашему желанию камера с возможностью приблизить и отдалить изображение.
- Фигуры прикрепляются к курсору мыши по щелчку.
- Расширяемый набор тестов.
- Графический интерфейс для выбранных тестов, настройка параметров и опция отладочной визуализации.
- Пауза и пошаговое моделирование.
- Визуализация текста.



Стенд содержит множество примеров использования Box2D. Загляните в исходный код тестового стенда, когда будете разбираться с Box2D.

Замечание: испытательный стенд написан с использованием freeglut и GLUI. Стенд не является частью библиотеки Box2D. Как показано в примере HelloWorld, вы можете использовать Box2D без какой либо графической визуализации результата.

Глава 3. Интерфейс Box2D

Содержание

3.1. Управление динамической памятью 3.2. Фабрики и определения 3.3. Единицы измерения 3.4. Пользовательские данные 3.5. Использование C++ 3.6. Недовольным

3.1. Управление динамической памятью

Необходимость в быстрой и эффективной работе с памятью внутри Box2D стала причиной большого количества дискуссий. В этом разделе рассказывается, как и зачем

Box2D выделяет память для своих нужд.

Box2D нуждается в выделении памяти для большого количества маленьких объектов (примерно по 50-300 байт). Использование системного хипа через функции `malloc` или `new` для небольших объектов неэффективно и приводит к фрагментации памяти. При этом бОльшая часть этих объектов имеет небольшое время жизни (например, контакты), но при этом может существовать на протяжении нескольких физических циклов. Поэтому необходим аллокатор (от англ. «allocator» - объект, занимающийся выделением памяти), который сможет эффективно обеспечивать нас динамической памятью для большого количества маленьких объектов.

В Box2D для решения этой проблемы используется "аллокатор маленьких объектов" («small object allocator» или сокращенно «SOA»). SOA хранит несколько блоков памяти различного размера. При требовании выделить память, SOA предоставляет нам блок памяти наилучшим образом соответствующий необходимому размеру. Если блок больше не нужен, то он просто помечается как свободный. Обе операции выполняются очень быстро и не приводят к фрагментации памяти.

Так как Box2D использует собственный SOA, НИКОГДА не выделяйте память для тел, фигур или соединений вручную (через malloc или new). Выделение памяти вручную разрешено только для b2World. Класс b2World предоставляет фабрики (специальные функции для создания объектов) для создания тел, фигур и соединений. Фабрики позволяют использовать эффективный механизм выделения памяти, при этом скрывая ненужные детали. Аналогично, НИКОГДА не используйте delete или free для тел, фигур и соединений.

При просчете текущего временного интервала, Box2D нуждается во временной рабочей памяти. Для этого он использует стековый аллокатор, чтобы каждый раз не обращаться к хипу. Необходимости во взаимодействии с этим аллокатором нет, просто достаточно знать, что он есть.

3.2. Фабрики и определения

Как упоминалось выше, управление памятью играет очень важную роль в устройстве интерфейса Box2D. Поэтому для создания b2Body или b2Joint, необходимо вызывать функции-фабрики класса b2World (за которыми скрывается быстрый и эффективный аллокатор памяти).

Вот эти функции:

```
 b2Body* b2World::CreateBody(const b2BodyDef* def)
b2Joint* b2World::CreateJoint(const b2JointDef* def)
```

А вот соответствующие функции для освобождения памяти:

```
 void b2World::DestroyBody(b2Body* body)
void b2World::DestroyJoint(b2Joint* joint)
```

При создании тела или соединения, необходимо передавать в функцию-фабрику указатель на определение тела (definition). Определения содержат всю информацию

необходимую фабрике для создания тела или соединения. Использование определений позволяет предотвратить ошибки при создании тел и соединений, сократить количество аргументов у функций-фабрик и уменьшить число аксессоров (функций для доступа к членам класса).

Так как любая фигура должна быть присоединена к телу-родителю, то и создаются они фабричными методами тела-родителя, т.е. класса `b2Body`:

```
 b2Shape* b2Body::CreateShape(const b2ShapeDef* def)
void b2Body::DestroyShape(b2Shape* shape)
```

Фабрики не сохраняют указатели на определения тел или соединений, поэтому определения можно объявлять как локальные переменные.

3.3. Единицы измерения

Так как Box2D использует числа с плавающей точкой, необходимо сразу допустимые размеры и отклонения от них. Параметры Box2D подобраны для использования единиц МКС (метр-килограмм-секунда) и работы с динамичными объектами, размер которых составляет от 0.1 до 10 метров, т.е. от стаканов до автобусов.

Идея использовать пиксели в качестве единиц измерения очень заманчива, но, к сожалению это приведет к неточному моделированию и, возможно, очень странному поведению объектов. Попробуйте представить моделирование движения высотных зданий движком, который был настроен на работу с куклами и бочками. Получится явно не очень хорошо.

> ***Внимание***
> Box2D использует единицы МКС. Используйте размеры движущихся объектов, в интервале от 0.1 до 10 метров. Вам будет нужно воспользоваться масштабированием при выводе результата в графическом виде. В примерах Box2D используется OpenGL-преобразование viewport.

3.4. Пользовательские данные

Классы `b2Shape`, `b2Body` и `b2Joint` позволяют сохранить указатель на какие-то другие данные в виде указателя на произвольный тип void. Это удобно, когда при использовании объектов Box2D необходимо задавать соответствующие им «игровые» объекты.

Например, часто этот указатель хранит ссылку на соответствующий «игровой объект». Получается своеобразная циклическая ссылка, когда, имея игровой объект, можно получить указатель на физическое тело и, наоборот, имея тело получить указатель на игровой объект.

```
GameActor* actor = GameCreateActor();
b2BodyDef bodyDef;
bodyDef.userData = actor;
```

```
actor->body = box2Dworld->CreateBody(&bodyDef);
```
Например, в нижеприведенных ситуациях без использования такого указателя не обойтись:

- Повреждение объекта после столкновения
- Выполнение какого-либо события при попадании персонажа в ограничивающий бокс
- Уведомление игровых объектов о разрыве соединения между физическими объектами

Следует помнить, что указатель на данные можно использовать по своему усмотрению и хранить в нем все что угодно, но при его использовании следует быть осторожным. Например, если в этом указателе хранится ссылка на игровой объект, то и в остальных объектах должна храниться ссылка того же типа, иначе, может произойти исключительная ситуация при попытке приведения типа указателя.

3.5. Использование C++

C++ хорош инкапсуляцией данных и полиморфизмом, но он не настолько хорош для построения API (интерфейсов программирование приложений).

Следовало ли нам воспользоваться концепцией абстрактных фабрик или какой-либо другой? В случае с фабриками, с одной стороны, API выглядит проще, но в конечном итоге мы встаем на путь эффективной разработки и отладки.

Следовало ли нам требовать использования концепции защищенных (private) данных и "дружественных" функций? Возможно, но со временем количество "друзей" могло стать огромным.

Может быть нам следовало обернуть код на C++ интерфейсами на Си? Может быть, но это огромная работа, которая к тому же может привести к неоптимальным изменениям внутри Box2D.

Для Box2D был выбран путь минимального сопротивления. Если для некоторых случаев классы с их устройством и функциональностью подходили лучше, то использовались открытые функции-интерфейсы и защищенные данные. В других случаях использовались классы или структуры, все данные которых были открыты для доступа извне. Благодаря такому подходу удалось достичь высокой скорости разработки, легкости при отладке и минимальной заботы о сохранении непроницаемой для стороннего взгляда инкапсуляции данных. Обратной стороной монеты стал тот факт, что вы не увидите чистого и простого API. Правда у вас есть это прекрасное руководство которое вам поможет :)

3.6. Недовольным

Не нравится как устроен API? Прекрасно! У вас есть исходный код! Серьезно, если у вас есть что сказать по поводу Box2D, пожалуйста, оставьте комментарий на форуме: http://www.box2d.org/forum

Глава 4. Мир

Содержание

4.1. О мире 4.2. Создание и уничтожение мира Box2D 4.3. Использование мира 4.3.1. Моделирование 4.3.2. Просмотр объектов мира 4.3.3. AABB запросы

4.1. О мире

Класс `b2World` содержит тела и соединения. Он управляет всеми аспектами моделирования и позволяет обрабатывать асинхронные очереди (как очереди AABB). Значительная часть вашего взаимодействия с Box2D будет происходить через объект `b2World`.

4.2. Создание и уничтожение мира Box2D

Создать мир совсем несложно. Необходимо лишь предоставить ограничивающий бокс мира (AABB) и вектор гравитации.

AABB должен содержать весь мир. Можно улучшить производительность, сделав бокс немного больше чем мир, например, раза в 2 для надежности. Если у вас много тел попадающих за пределы мира, то эти тела необходимо находить и удалять. Это повысит производительность и предотвратит возможное переполнение при обработке чисел с плавающей точкой. Для создания и удаления мира следует использовать соответственно операторы new и delete.

```
b2World* myWorld = new b2World(aabb, gravity, doSleep);
... сделать что-нибудь ...
delete myWorld;
```

> ***Внимание***
> Еще раз повторяем, AABB мира всегда должен быть больше региона, где располагаются тела. Если тело (возможно в процессе моделирования) пересекает границу AABB мира, то оно замораживается и исключается из процесса моделирования.

4.3. Использование мира

Класс мира содержит функции-фабрики, предназначенные для создания и уничтожения

тел и соединений. Эти функции будут рассмотрены далее в разделах "Тела" (bodies) и "Соединения" (joints). С `b2World` возможны еще несколько операций (кроме создания и удаления тел и соединения), эти операции мы сейчас и рассмотрим.

4.3.1. Моделирование

Класс мира используется для запуска и управления процессом моделирования. Вы задаете длительность шага и количество итераций на шаг. Например, так:

```
 float32 timeStep = 1.0f / 60.f;
 int32 iterationCount = 10;
myWorld->Step(timeStep, iterationCount);
```

После выполнения расчета шага физики можно обработать все тела и соединения. Скорее всего, вы будете проверять позиции тел, чтобы иметь возможность обновить позиции объектов в игре и прорисовать их на экране. Можно выполнять расчет шага физики в любом месте игрового цикла, но следует быть осторожным с порядком выполнения операций. Например, если необходимо в текущем кадре создать новое тело и сразу проверить пересечение объектов с ним, то нужно сначала создать тело, расположить его в нужном месте мира, запустить расчет шага физики, и только потом проверять произошло ли столкновение.

Как уже обсуждалось в уроке `HelloWorld`, следует использовать фиксированную длительность шага. Использование больших шагов повысит производительность в приложениях с низкой частотой кадров. Но не стоит использовать шаг длиннее 1/30 секунды. Обычно 1/60 секунды достаточно для высококачественного моделирования.

Количество итераций определяет, сколько раз будут произведены расчеты над всеми телами и соединениями в мире. Большее число итераций всегда обеспечивает лучшее качество моделирования. Но не стоит сочетать маленький шаг с большим количеством итераций. 60Гц (Герц) и 10 итераций гораздо лучше 30Гц и 20 итераций.

Примечание переводчика

Дело здесь в том, что при делении очень маленького числа (длины шага) на очень большое (кол-во итераций) мы кроме всего прочего можем вообще ноль получить.

Пример погрешности при работе с маленькими числами:

```
float A = 0.0001f;
int   B = A*10000;
```

В MSVS2008 на стандартных настройках B равно 0 а совсем не 1.

kkray

4.3.2. Просмотр объектов мира

Как упоминалось ранее, мир является "контейнером" для всех объектов и соединений. Поэтому, из мира можно получить список всех тел и соединений мира, а затем сделать что-нибудь с каждым из них. Например, следующий код заставляет все тела "проснуться":

```
for (b2Body* b = myWorld->GetBodyList(); b; b = b->GetNext())
{
```

```
	b->WakeUp();
}
```

К сожалению, в жизни не все так просто. Например, следующий вариант кода будет работать неправильно:

```
for (b2Body* b = myWorld->GetBodyList(); b; b = b->GetNext())
{
	GameActor* myActor = (GameActor*)b->GetUserData();
	if (myActor->IsDead())
	{
		// ОШИБКА: после этой операции GetNext возвратит мусор
		myWorld->DestroyBody(b);
	}
}
```

В этом коде все будет работать правильно только до момента удаления тела. После удаления тела, указатель на следующее тело (который хранится внутри тела) станет неправильным. Поэтому функция b2Body::GetNext() вернет указатель на мусор. Во избежание этого, необходимо сохранять указатель на следующее тело перед удалением, как в примере ниже.

```
b2Body* node = myWorld->GetBodyList();
while (node)
{
	b2Body* b = node;
	node = node->GetNext();

	GameActor* myActor = (GameActor*)b->GetUserData();
	if (myActor->IsDead())
	{
		myWorld->DestroyBody(b);
	}
}
```

Данный код производит безопасное удаление текущего тела. Однако, может потребоваться функция удаляющая сразу несколько тел. В таком случае надо быть осторожным. Пример такой функции приведен ниже, но точный вид функции будет зависеть от вашего приложения.

```
b2Body* node = myWorld->GetBodyList();
while (node)
{
	b2Body* b = node;
	node = node->GetNext();

	GameActor* myActor = (GameActor*)b->GetUserData();
	if (myActor->IsDead())
	{
		bool otherBodiesDestroyed = GameCrazyBodyDestroyer(b);
		if (otherBodiesDestroyed)
		{
			node = myWorld->GetBodyList();
		}
	}
}
```

Здесь, функция GameCrazyBodyDestroyer производит удаление тела и/или других тел,

поэтому, если удаляется несколько тел, то указатель node на следующее тело может оказаться неправильным и надо начинать просмотр тел с начала.

4.3.3. AABB запросы

Иногда требуется определить все фигуры находящиеся в определенном регионе. Класс `b2World` содержит такую функцию и скорость ее работы высока. Функция принимает в качестве параметров AABB в мировых координатах и возвращает массив всех фигур, которые пересекаются с AABB (или находятся полностью в нём). Слово "пересекаются" не совсем правильно, так как проверка на пересечение происходит между AABB региона и AABB самих фигур. Например, следующий код находит все тела находящиеся в AABB и будит их.

```
 b2AABB aabb;
 aabb.lowerBound.Set(-1.0f, -1.0f);
 aabb.upperBound.Set(1.0f, 1.0f);
 const int32 k_bufferSize = 10;
 b2Shape *buffer[k_bufferSize];

 int32 count = myWorld->Query(aabb, buffer, k_bufferSize);
 for (int32 i = 0; i < count; ++i)
 {
       buffer[i]->GetBody()->WakeUp();
}
```

Глава 5. Тела

Содержание

5.1. О телах 5.2. Определение тела 5.2.1. Масса 5.2.2. Позиция и угол 5.2.3. Торможение 5.2.4. Параметры сна 5.2.5. Снаряды 5.3. Фабрика тел 5.4. Использование тел 5.4.1. Данные о массе 5.4.2. Информация о состоянии тела 5.4.3. Положение и скорость 5.4.4. Силы и импульсы 5.4.5. Преобразование координат 5.4.6. Списки

5.1. О телах

У тел есть положение и скорость. К ним можно применять силы, моменты и импульсы. Тела могут быть статичными или динамичными, статичные тела никогда не перемещаются и не взаимодействуют с другими статичными телами.

Тела — основа всех игровых форм. Они состоят из фигур и перемещаются вместе с ними. Тела прежде всего являются "твердыми". Это означает, что две фигуры, присоединенные к одному телу, всегда будут оставаться неподвижными друг для друга.

В "игровом" движке обычно хранятся указатели на все созданные физические тела. Поэтому, при необходимости, можно получить положение тела, например, для обновления графического объекта. Указатели на тела также необходимо сохранить, если их потребуется удалить.

5.2. Определение тела

Перед созданием тела, необходимо заполнить определение тела (`b2BodyDef`). `b2BodyDef` можно задать как в качестве локальной переменной, так и в качестве части структуры игровых данных. Выбор за вами.

Box2D копирует определение тела и не хранит указатель на полученное определение. Поэтому одно определение можно использовать для создания нескольких тел.
Давайте рассмотрим основные части структуры `b2BodyDef`.

5.2.1. Масса

Есть несколько способов задания массы тела:
1. Задать значение массы непосредственно в определении тела.
2. Задать значение массы уже для тела, после его создания.
3. Вычислить массу, основываясь на плотности присоединенных фигур.

В большинстве игр имеет смысл вычислять массу, учитывая плотность фигур. Это позволяет проверить, имеют ли тела реалистичную массу.

Однако, иногда может потребоваться конкретное значение массы. Например, в механизме наподобие весов, который требует точные значения масс.
Можно вручную установить массу в определении тела:

```
 bodyDef.massData.mass = 2.0f; // масса тела в кг
 bodyDef.center.SetZero(); // центр массы в локальных координатах тела
bodyDef.I = 3.0f; // момет инерции в кг*м^2
```

Далее в руководстве будут использованы и другие способы задания массы.

5.2.2. Позиция и угол

Определение тела позволяет указать начальную позицию тела. Это быстрее, чем создавать тело, а затем перемещать его.

Тело имеет два важных параметра. Первый — позиция тела. Фигуры и соединения создаются в координатах относительно позиции тела. Второй — центр масс. Позиция центра масс определяется исходя из плотности фигур и их формы или непосредственно параметром `massData` определения тела. Большинство внутренних вычислений Box2D происходит с использованием центра масс. Например, `b2Body` хранит линейную скорость именно центра массы. При заполнении определения тела, может быть неизвестно, где находится центр масс, поэтому указывается позиция тела. Также можно указать угол поворота тела в радианах, который не зависит от позиции центра масс. Изменение

позиции центра масс не влияет на позицию тела и присоединенные фигуры и соединения.

```
 bodyDef.position.Set(0.0f, 2.0f); // позиция тела
bodyDef.angle = 0.25f * b2_pi; // угол поворота тела в радианах
```

5.2.3. Торможение

Торможение используется, чтобы уменьшать скорость тел. Оно отличается от трения тем, что трение уменьшает скорость только при касании, а торможение действует постоянно. Таким образом, торможение не заменяет трение и они должны использоваться вместе. Параметр "торможение" должен иметь значение между нулем и бесконечностью, где 0 - отсутствие торможения, а бесконечность — полное торможение. Чаще всего используется "торможение" из диапазона 0 — 0.1. Линейное торможение обычно не используется, т.к. оно делает тела похожими на находящиеся под водой.

```
 bodyDef.linearDamping = 0.0f;
bodyDef.angularDamping = 0.01f;
```

При маленьком значении торможения эффект по большей части не зависит от величины временного шага, но при больших значениях будет заметно зависеть. Это не станет проблемой, при использовании фиксированной длины шага (как рекомендуется).

5.2.4. Параметры сна

Для чего нужен сон? Дело в том, что моделирование тел довольно дорогая задача и чем меньше объектов участвует в моделировании, тем лучше. Когда тело переходит в состояние покоя необходимо прекратить для него моделирование.

Когда тело (или группа тел) переходит в состояние покоя, Box2D переводит это тело в состояние сна, которое очень слабо загружает процессор. Если бодрствующее тело касается спящего, то спящее тело просыпается. Тела могут также просыпаться если разрушается соединение или контакт, присоединенный к нему. Также тело можно разбудить вручную.

Определение тела позволяет вам точно определить, может ли тело *засыпАть* и создается ли тело уже спящим.

```
 bodyDef.allowSleep = true;
bodyDef.isSleeping = false;
```

5.2.5. Снаряды

Игровое моделирование обычно генерирует последовательность изображений-кадров, которые показываются с определенной частотой (Frames Per Second, FPS). В таких условиях твердые тела могут передвигаться на большие расстояния за один шаг

расчетов. Если физический движок не предназначен для расчетов больших передвижений, то можно увидеть как одни объекты проходят сквозь другие. Этот эффект получил название *туннельного.*

По умолчанию, Box2D использует определение столкновений на протяжении пути (continuous collision detection, CCD) чтобы предохранить динамические объекты от проскакивания сквозь статичные объекты, т.е. от туннельного эффекта. Работает это очень просто: фигуры протягиваются от начальной до конечной позиции. Движок следит, не происходит ли столкновений в процессе перемещения тела между позициями и определяет время столкновения (time of impact,TOI). Тела перемещаются в точки соответствующие их первым TOI и моделирование продолжается до конца временного отрезка. Этот процесс повторяется столько раз, сколько понадобится.

Обычно CCD не используется для расчетов столкновений между динамичным телами. Это сделано чтобы сохранить разумную скорость работы. В некоторых игровых ситуациях вам может понадобится использовать CCD для динамичных тел. Например вы можете захотеть выстрелить высокоскоростной пулей в тонкую стену. Без CCD, пуля, скорее всего, пролетела бы сквозь стену.

Быстродвижущиеся объекты в Box2D именуются "снарядами" (от англ. *bullet* — пуля, ядро).

```
bodyDef.isBullet = true;
```
Установка флага "снаряда" действенна только для динамичных тел.

CCD это дорогая операция так что, лучше не отмечать все движущиеся тела как "снаряды". Так в Box2D по умолчанию CCD используется только между парами динамичное тело - статичное тело. Этот подход эффективен, если необходимо не дать телам покинуть ваш игровой мир. Однако вы можете иметь несколько быстродвижущихся объектов которые потребуют постоянного использования CCD.

5.3. Фабрика тел

Тела создаются и уничтожаются с использованием методов-фабрик объекта `b2World`. Это позволяет миру создать тело с использованием эффективного аллокатора (SOA) и добавить его в мир.

Тела могут быть динамичными или статичными в зависимости от массы. Оба типа тел создаются и уничтожаются при помощи одних и тех же методов.

```
b2Body* dynamicBody = myWorld->CreateBody(&bodyDef);
... сделать что-нибудь ...
myWorld->DestroyBody(dynamicBody);
     dynamicBody = NULL;
```

> ***Внимание***
> Никогда не используйте new или malloc для создания тел. Мир не будет знать о существовании этого тела и оно не будет правильно настроено.

Статические тела не перемещаются под воздействием других тел. Можно вручную

перемещать статичные тела, но следует быть осторожным, чтобы не раздавить динамичные тела, находящиеся между двумя или несколькими статичными. Трение не работает при перемещении статичного тела, а также статичные тела никогда не взаимодействуют друг с другом. При создании статичной геометрии игрового уровня, быстрее подсоединить несколько фигур к одному статичному телу, чем создавать несколько статичных тел. Box2D устанавливает массу и обратную массу статичных тел равными нулю, поэтому для внутренних алгоритмов моделирования нет разницы между статичными и динамичными телами.

Box2D не хранит ссылки на определение тела или любые находящиеся в нем данные (за исключением указателя на пользовательские данные). Поэтому можно объявлять определения как локальные переменные и использовать одно и то же определение для создания нескольких тел.

Box2D избавляет вас от необходимости удалять каждого тело вручную, т.к. при удалении мира, он (мир) сам выполняет всю работу по сборке мусора. Однако после этого необходимо обнулить все указатели на физические тела в игровом движке.

При удалении тела, все присоединённые к нему фигуры и соединения удаляются автоматически. Эта деталь имеет большое значение при использовании указателей на фигуры и соединения. Более подробная информация доступна в 9.2 Неявное уничтожение.

Допустим необходимо соединить динамическое тело с неподвижной точкой. Для этого необходимо соединить динамичное тело со статичным телом. Если статичного тела нет, то можно использовать общий бокс земли.

```
 b2Body* ground = myWorld->GetGroundBody();
... создать соединение используя статичное тело земли ...
```

5.4. Использование тел.

После создания тела, над ним можно производить множество операций. Включая настройку параметров массы, получение позиции и скорости тела, приложение к нему сил, преобразование точек и векторов между локальными и мировыми координатами.

5.4.1. Данные о массе

Можно задавать массу прямо во время выполнения программы. Обычно, это делается когда создается или уничтожается фигура тела. Может потребоваться задать массу тела исходя из данных о присоединенных к нему фигурах.

```
void SetMassFromShapes();
```

Также бывает необходимо установить массу для тела вручную. Например, если необходимо изменить фигуры, но при этом использовать свои собственные формулы для расчета массы.

```
void SetMass(const b2MassData* massData);
```

Данные о массе тела доступны через следующие функции:

```
 float32 GetMass() const;
 float32 GetInertia() const;
const b2Vec2& GetLocalCenter() const;
```

5.4.2. Информация о состоянии тела

Состояние тела имеет много аспектов. Можно получить доступ к данным о состоянии тела, используя следующие функции:

```
 bool IsBullet() const;
 void SetBullet(bool flag);

 bool IsStatic() const;
 bool IsDynamic() const;

 bool IsFrozen() const;

 bool IsSleeping() const;
 void AllowSleeping(bool flag);
void WakeUp();
```

Состояние снаряда описано в разделе 5.2.5, “Снаряды”. Состояние замороженности описано в разделе 9.1, “Границы мира”.

5.4.3. Положение и скорость

Обычно необходимо узнать позицию и угол поворота физического тела, например для отображения соответствующего графического объекта. Можно вручную установить позицию тела, но так делают редко, обычно для моделирования перемещений используется Box2D.

```
 bool SetXForm(const b2Vec2& position, float32 angle);
 const b2XForm& GetXForm() const;
 const b2Vec2& GetPosition() const;
float32 GetAngle() const;
```

Можно узнать мировые координаты центра масс тела. В большинстве внутренних алгоритмов Box2D для моделирования используется именно центр масс. Но обычно знать координаты центра масс не требуется, чаще нужно знать позицию тела.

`const b2Vec2& GetWorldCenter() const;`Можно получить доступ к линейной и угловой скорости тела. Линейная скорость устанавливается для центра масс.

```
 void SetLinearVelocity(const b2Vec2& v);
 b2Vec2 GetLinearVelocity() const;
 void SetAngularVelocity(float32 omega);
float32 GetAngularVelocity() const;
```

5.4.4. Силы и импульсы

К телу можно применять силы, моменты и импульсы. При применении силы или импульса указывается мировая координата их приложения. Часто при этом возникает момент вращения вокруг центра масс.

```
 void ApplyForce(const b2Vec2& force, const b2Vec2& point);
 void ApplyTorque(float32 torque);
void ApplyImpulse(const b2Vec2& impulse, const b2Vec2& point);
```

Применение силы, момента или импульса выводит тело из состояния сна. Иногда требуется применить силу только для бодрствующих тел, для этого случая подойдет следующий код:

```
 if (myBody->IsSleeping() == false)
 {
      myBody->ApplyForce(myForce, myPoint);
}
```

5.4.5. Преобразование координат

Класс b2Body имеет вспомогательные функции для преобразования точек и векторов между локальными и мировыми координатами. Если вам неясны эти понятия, то можно ознакомиться с книгой "Essential Mathematics for Games and Interactive Applications" авторов Jim Van Verth и Lars Bishop. Эти функции не ресурсоёмкие, так что ими можно смело пользоваться.

```
 b2Vec2 GetWorldPoint(const b2Vec2& localPoint);
 b2Vec2 GetWorldVector(const b2Vec2& localVector);
 b2Vec2 GetLocalPoint(const b2Vec2& worldPoint);
b2Vec2 GetLocalVector(const b2Vec2& worldVector);
```

5.4.6. Списки

При желании можно получить доступ ко всем фигурам заданного тела. Что очень удобно если необходимо получить пользовательские данные для каждой фигуры присоединенной к определенному телу.

```
 for (b2Shape* s = body->GetShapeList(); s; s = s->GetNext())
 {
      MyShapeData* data = (MyShapeData*)s->GetUserData();
      ... сделать что-то ...
}
```

Таким же образом можно пройтись по всем соединениям тела.

Глава 6. Фигуры

Содержание

6.1. О фигурах 6.2. Определение фигуры 6.2.1. Коэффициенты трения и восстановления 6.2.2. Плотность 6.2.3. Фильтрация столкновений 6.2.4. Сенсоры 6.2.5. Определение Окружности 6.2.6. Определение Полигона 6.3. Фабрика фигур 6.4. Использование фигур

6.1. О фигурах

Фигуры это геометрия, которая подсоединяется к телам и используется для проверки столкновений между ними. Фигуры также используются для определения массы тела. Это позволяет определить массу тела лишь задав плотность, Box2D сделает все вычисления сам.

Фигуры имеют коэффициенты трения и упругости. В фигурах содержится информация о фильтрах столкновений, что позволяет не проверять столкновения между определенными игровыми объектами (скажем можно сделать стенку через которую можно будет стрелять но нельзя будет пройти -- прим. перев.)

Фигуры всегда принадлежат определенному телу. К одному телу можно присоединить несколько фигур. Фигуры являются абстрактными классами, поэтому в Box2D могут быть реализованы различные их типы. Поэтому при желании можно создать свой собственный тип фигур и соответствующие алгоритмы расчета столкновений между ними.

6.2. Определение фигуры

Определения фигур используются для создания фигур. В определении b2ShapeDef присутствует как общая для всех информация, так и специфичная информация для отдельных типов фигур.

6.2.1. Коэффициенты трения и упругости

Трение используется для создания реалистичного скольжения одних объектов по другим. Box2D поддерживает статическое и динамичное трение, но использует один и тот же параметр для обоих. Трение в Box2D моделируется безошибочно, и сила трения пропорционально зависит от величины нормальной силы (трение Колумба). Параметр трения имеет значение в пределах от 0 до 1. Ноль соответствует отсутствию трения, а единица — максимальному трению. При вычислении трения между двумя телами, Box2D должен использовать коэффициенты трения обоих тел. Результирующее трение вычисляется по следующей формуле:

```
 float32 friction;
friction = sqrtf(shape1->friction * shape2->friction);
```

Коэффициент упругости используется при столкновении тел. Значение коэффициента может быть из диапазона от 0 до 1. Например, при падении шара на стол, значение 0 означает, что шар не отскочит - это называется не упругое столкновение, при единице же шар отскочит с той же скоростью -- это абсолютно упругое столкновение. При столкновении двух тел результирующий коэффициент вычисляется по следующей формуле:

```
 float32 restitution;
```

```
restitution = b2Max(shape1->restitution, shape2->restitution);
```
Если тела имеют несколько точек соприкосновения, то восстановление моделируется приближенно. Это происходит из-за того, что Box2D использует итеративный "разрешальщик" столкновений. Box2D также использует не эластичные столкновения, если скорость столкновения мала. Это необходимо для предотвращения дрожания тел.

6.2.2. Плотность

При необходимости Box2D может вычислить массу и момент инерции тел, используя плотность присоединенных фигур. Указание массы тела вручную, часто приводит к плохому моделированию, поэтому, лучшим путём задания массы тела будет сначала указание плотностей всех фигур тела, а затем вызов b2Body::SetMassFromShape.

6.2.3. Фильтрация столкновений

Фильтрация столкновений позволяет предотвратить столкновения между определенными фигурами. Например, вы создаете персонажа, который едет на велосипеде. Вам необходимо, чтобы велосипед и персонаж взаимодействовали с ландшафтом, но при этом велосипед и персонаж не взаимодействовали между собой (так как они пересекаются). Для таких случаев Box2D поддерживает фильтрацию столкновений при помощи категорий и групп.

Box2D поддерживает 16 категорий взаимодействия. Для каждой фигуры, можно указать категорию, которой она принадлежит. Можно также указать с какими другими категориями может взаимодействовать эта фигура. Например, в многопользовательской игре вам необходимо указать, что все игроки не взаимодействуют друг с другом, и что монстры не взаимодействуют друг с другом, но при этом необходимо, чтобы игроки и монстры взаимодействовали друг с другом. Это можно осуществить при помощи "маски битов" (maskBits). Например:
```
 playerShapeDef.filter.categoryBits = 0x0002;
 monsterShapeDef.filter.categoryBits = 0x0004;
 playerShape.filter.maskBits = 0x0004;
monsterShapeDef.filter.maskBits = 0x0002;
```

Группы взаимодействия позволяют указать индекс группы. Все фигуры с одинаковым индексом группы могут всегда взаимодействовать (если индекс положительный) или никогда не взаимодействовать (если индекс отрицательный). Индексы групп обычно используются для вещей которые как-нибудь связаны, подобно частям велосипеда. В следующем примере, shape1 и shape2 всегда взаимодействуют, а shape3 и shape4 - никогда.
```
 shape1Def.filter.groupIndex = 2;
 shape2Def.filter.groupIndex = 2;
 shape3Def.filter.groupIndex = -8;
shape4Def.filter.groupIndex = -8;
```
Столкновения между фигурами различных групп фильтруются в соответствии с категориями и "маскирующими" битами. Другими словами, группы имеют более высокий приоритет по сравнению с категориями.
Следует отметить, что в Box2D происходит дополнительная фильтрация столкновений:

- Фигуры статичных тел никогда не взаимодействуют друг с другом.
- Фигуры одного и того же тела никогда не взаимодействуют друг с другом.
- При желании можно указать будут ли сталкиваться фигуры соединенных тел.

Иногда необходимо изменить фильтрацию столкновений после того как фигура была создана. Можно получить и установить структуру `b2FilterData` существующей фигуры, используя методы b2Shape::GetFilterData и b2Shape::SetFilterData. Box2D кэширует результаты фильтрования, поэтому, после изменения фильтрации, нужно вызвать b2World::Refilter.

6.2.4. Сенсоры

Иногда требуется узнать когда пересеклись две фигуры и при этом не требуется обрабатывать столкновение между ними. Эта задача решается при помощи сенсоров. Сенсор — это фигура, которая лишь обнаруживает столкновения с другими телами и при этом не влияет на них.
Можно пометить любую фигуру (статичную и динамичную) как сенсор. При этом если тело имеет несколько фигур, то любые из них могут быть сенсорами.

```
myShapeDef.isSensor = true;
```

6.2.5. Определение Окружности

b2CircleDef это расширение b2ShapeDef путем добавления полей радиуса и локальной позиции.

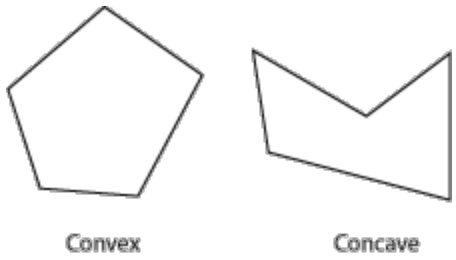
```
 b2CircleDef def;
 def.radius = 1.5f;
def.localPosition.Set(1.0f, 0.0f);
```

6.2.6. Определение Полигона

Структура `b2PolyDef` используется для создания выпуклых полигонов. Создание полигонов имеет нюансы, поэтому внимательно читайте текст. Максимальное число вершин определяется константой `b2_maxPolygonVertices`, по умолчанию равной 8. Если необходимо использовать большее число вершин, то надо изменить константу `b2_maxPolygonVertices` в файле `b2Settings.h` .

При построении полигона необходимо указать число вершин, которое будет использоваться. Координаты вершин должны быть указаны в порядке против часовой стрелки относительно оси z правосторонней системы координат. Но на экране это может выглядеть как по часовой стрелке в зависимости от экранных координат.

Полигоны должны быть выпуклыми и грани полигона не должны пересекаться. Другими словами, каждая вершина должна выглядывать наружу. Box2D автоматически замкнёт контур полигона.



Пример определения полигона в виде треугольника:

```
 b2PolygonDef triangleDef;
 triangleDef.vertexCount = 3;
 triangleDef.vertices[0].Set(-1.0f, 0.0f);
 triangleDef.vertices[1].Set(1.0f, 0.0f);
triangleDef.vertices[2].Set(0.0f, 2.0f);
```

Вершины задаются в координатной системе тела. Если необходимо сместить полигон относительно тела, то надо только сместить позиции вершин.
Для удобства, имеются функции инициализации полигонов в виде боксов. Можно создать как AABB с центром в позиции тела, так и повернутый бокс со смещением относительно позиции тела.

```
 b2PolygonDef alignedBoxDef;
 float32 hx = 1.0f; // половина ширины
 float32 hy = 2.0f; // половина высоты
 alignedBoxDef.SetAsBox(hx, hy);

 b2PolygonDef orientedBoxDef;
 b2Vec2 center(-1.5f, 0.0f);
 float32 angle = 0.5f * b2_pi;
orientedBoxDef.SetAsBox(hx, hy, center, angle);
```

6.3. Фабрика фигур

Для того чтобы создать фигуру надо заполнить определение фигуры и передать его методу-фабрике тела.

```
 b2CircleDef circleDef;
 circleDef.radius = 3.0f;
 circleDef.density = 2.5f;
 b2Shape* myShape = myBody->CreateShape(&circleDef);
[если необходимо, то используем указатель на фигуру]
```

Этот код создает фигуру и подсоединяет её к телу. Указатель на фигуру необязательно где-то хранить, так как фигура автоматически уничтожится при уничтожении тела (9.2 Неявное уничтожение).

После подсоединения фигур к телу, может потребоваться пересчет массы тела.

```
myBody->SetMassFromShapes();
```

Эта функция довольно ресурсоемкая, поэтому вызывать её следует только при необходимости.

Определенная фигура тела удаляется просто. Это можно использовать для моделирования разрушающихся объектов. Надо просто попросить тело удалить

присоединенную фигуру.
```
myBody->DestroyShape(myShape);
```
После уничтожения фигур тела, желательно вызвать метод SetMassFromShapes снова.

6.4. Использование фигур

Здесь можно сказать не так уж много. Можно узнать тип фигуры и какому телу она принадлежит, можно проверить находится ли заданная точка в фигуре. Обо всех возможностях можно узнать просмотрев файл b2Shape.h .

Глава 7. Соединения

Содержание

7.1. О соединениях 7.2. Определение соединения 7.2.1. Соединение "жесткий отрезок" 7.2.2. Болтовое соединение 7.2.3. Призматическое соединение 7.2.4. Талевое соединение 7.2.5. Передаточное соединение 7.2.6. Соединение с курсором мыши 7.3. Фабрика соединений 7.4.Использование соединений 7.4.1. Использование соединений "жесткий отрезок" 7.4.2. Использование болтовых соединений 7.4.3. Использование призматических соединений 7.4.4. Использование талевых соединений 7.4.5. Использование передаточных соединений7.4.6. Использование соединений с курсором мыши

7.1. О соединениях

Соединения используются для присоединения тела к миру или другим телам. Типичный игровой пример рэгдолы, качели и подъемные тали. Соединения могут сочетаться множеством способов, что позволяет получить множество интересных движений.

Некоторые соединения предоставляют ограничения, при помощи которых вы можете контролировать диапазон возможный движений этого соединения. Некоторые предоставляют двигатели-мускулы, которые можно использовать для приведения соединения в движение с заданной скоростью и силой/вращающим моментом.

Соединения-"мускулы" могут использоваться множеством способов. Задавая соединению скорость пропорциональную расстоянию от текущей точки до желаемой можно двигать предметы. Вы также можете использовать "мускулы" для симуляции амортизирующего, трущегося соединения: задайте соединению нулевую скорость, и в качестве максимально возможной силы/вращающего момента соединения задайте маленькое значение. Тогда двигатель будет пытаться сохранить соединение неподвижным, пока нагрузка не станет слишком высокой.

7.2. Определение соединения

Все определения соединений унаследованы от `b2JointDef`. Все соединения создаются между двумя различными телами. Одно из них может быть статическим. Впрочем, если хотите растрачивать помять, то можете соединять и статические тела тоже :)

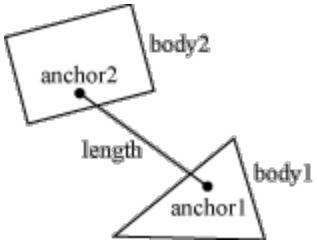
Вы можете задать пользовательские данные для любого соединения, а также можете отключить проверку столкновений между соединенными телами. На самом деле, это поведение по умолчанию, и если вы хотите проверять столкновения соединенных тел, то вам нужно вручную выставить флаг collideConnected в `true`.

Многие определения соединений требуют, чтобы вы предоставили некоторые геометрические данные. Часто соединения будут определяться через *точки-якори* (*anchor points).* Это точки, которые зафиксированы на присоединенных телах. Box2D требует, чтобы эти точки определялись в локальных координатах тел. В этом случае соединение может быть задано даже если трансформации тела насильно изменили соединение, например, когда игра сохраняется и перезагружается. Дополнительно, некоторые определения тел должны знать изначальный относительный угол между телами (болтовое соединение). Это необходимо в частности для корректного ограничения вращения, при использовании ограничений или фиксированого угла между телами. Т.к. инициализация геометрических данных может быть довольно скучным занятием, многие типы соединений имеют функции инициализации, которые используют текущую положение тела чтобы уменьшить количество требуемой от вас работы. Тем не менее, эти функции инициализации обычно следует использовать только для прототипирования. В конечном коде лучшее определять геометрию вручную. Это сделает поведение соединений более адекватным.

Содержимое определения тела зависит от типа соединения. Далее мы рассмотрим каждое из них.

7.2.1. Соединение "жесткий отрезок"

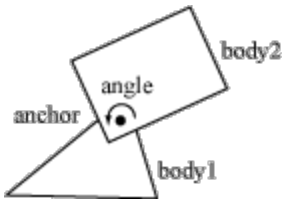
Самым простым типом соединения является соединение "жесткий отрезок", которое означает, что расстояние между двумя точками двух тел всегда должно быть постоянным. При создании соединения "жесткий отрезок" тела уже должны находиться в нужном положении. Затем указываются *якорные* точки в мировых координатах. Первая якорная точка присоединяется к первому телу, а вторая соответственно ко второму. Расстояние между точками подразумевает длину жесткого отрезка.
Пример определения соединения "жесткий отрезок". В данном случае мы разрешаем соединенным телам сталкиваться.




```
b2DistanceJointDef jointDef;
jointDef.Initialize(myBody1, myBody2, worldAnchorOnBody1,
worldAnchorOnBody2);
jointDef.collideConnected = true;
```

7.2.2. Болтовое соединение

Болтовое соединение можно представить, как будто в определенной точке мы просверлили отверстие через оба тела и вставили в него болт, вокруг которого они могут вращаться. Эту точку часто называют *шарнирной точкой.* Болтовое соединение имеет одну степень свободы: угол поворота второго тела относительно первого. Этот угол называется *угол соединения*.



Для создания болтового соединения необходимо предоставить два тела и указать якорную точку в мировых координатах. Функция инициализации предполагает что тела уже находятся в правильном положении.
В этом примере, два тела связываются болтовым соединением в точке центра масс первого тела.

```
b2RevoluteJointDef jointDef;
jointDef.Initialize(myBody1, myBody2, myBody1->GetWorldCenter());
```

Угол болтового соединения положителен, когда body2 поворачивается против часовой стрелки относительно body1. Подобно всем другим углам в Box2D, угол болтового соединения измеряется в радианах. По умолчанию, угол болтового соединения после создания равен нулю, независимо от текущих углов поворота тел.
Иногда требуется контролировать угол соединения. Для этого, болтовое соединение может моделировать ограничение и/или мускул. Ограничение соединения заставляет угол болтового соединения находиться в определенных пределах. При выходе угла болтового соединения за пределы к телам применяется момент для возврата угла в правильное положение. Поэтому верхний или нижний предел угла соединения должен включать ноль, иначе при начале моделирования будет заметен резкий рывок тел.
Мускул позволяет указать скорость соединения (производную угла по времени). Скорость может быть положительной или отрицательной. Мускул может иметь бесконечную силу, но это не желательно. Может вы когда-нибудь слышали такое:

Внимание
"Что произойдет, когда непреодолимая сила встретит непоколебимый объект?"

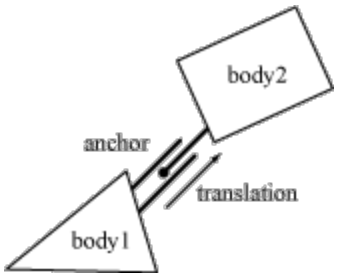
Могу сказать что ничего хорошего. Можно задать максимальный вращающий момент для мускула. Мускул будет поддерживать заданную скорость, пока необходимый для этого вращающий момент не превысит определённого максимума. При превышении максимума, вращение соединения будет приостановлено и может продолжиться в другом направлении. Можно использовать мускул для моделирования трения в соединении. Надо только установить скорость соединения в ноль, а максимальный вращающий момент сделать небольшой, но не нулевой величиной. Мускул будет пытаться предотвратить вращение соединения, но не будет способен противостоять значительной нагрузке.

Вот пример использования болтового соединения с учетом рассмотренных выше понятий; на этот раз соединение имеет предел и мускул. Мускул настроен на моделирование трения соединения.

```
 b2RevoluteJointDef jointDef;
 jointDef.Initialize(body1, body2, myBody1->GetWorldCenter());
 jointDef.lowerAngle = -0.5f * b2_pi; // -90 градусов
 jointDef.upperAngle = 0.25f * b2_pi; // 45 градусов
 jointDef.enableLimit = true;
 jointDef.maxMotorTorque = 10.0f;
 jointDef.motorSpeed = 0.0f;
jointDef.enableMotor = true;
```

7.2.3. Призматическое соединение

Призматическое соединение позволяет телам перемещаться лишь относительно определенной оси (заданной относительно тел) и не допускает вращения этих тел. Поэтому, призматическое соединение имеет одну степень свободы: перемещение второго тела относительно первого.



Определение призматического соединения подобно описанию болтового соединения; только заменяет угол перемещением, а момент силой. По аналогии с болтовым соединением, определение призматического соединения с ограничениями и мускулом

будет иметь вид:

```
 b2PrismaticJointDef jointDef;
 b2Vec2 worldAxis(1.0f, 0.0f);
 jointDef.Initialize(myBody1, myBody2, myBody1->GetWorldCenter(),
 worldAxis);
 jointDef.lowerTranslation = -5.0f;
 jointDef.upperTranslation = 2.5f;
 jointDef.enableLimit = true;
 jointDef.motorForce = 1.0f;
 jointDef.motorSpeed = 0.0f;
jointDef.enableMotor = true;
```

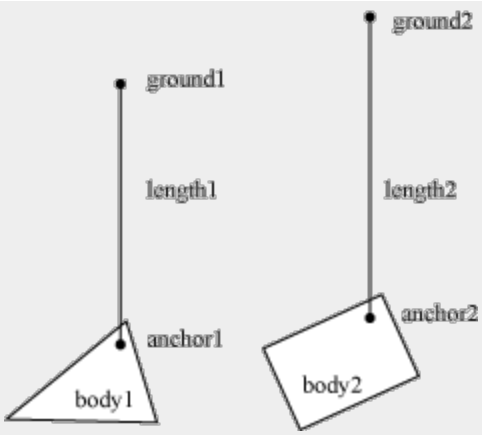
Болтовое соединение имеет неявно задаваемую ось (которая перпендикулярна экрану, поэтому не может быть отображена) вокруг которой тела могут вращаться. Для призматического соединения необходимо явно указать ось лежащую в плоскости экрана. Эта ось сохраняет своё положение относительно тел и перемещается вместе с ними.

Подобно болтовому соединению, перемещение призматического соединения равно нулю при создании соединения (вызов метода Initialize()). Поэтому необходимо чтобы верхний или нижний предел был равен нулю.

7.2.4. Талевое соединение

Талевые соединения используются для создания идеальных талей. Тали соединяют два тела с землей и друг с другом. Общая длина двух сегментов талей фиксирована и удовлетворяет следующему уравнению.

```
length1 + length2 == constant
```



Вы можете использовать коэффициент, чтобы смоделировать "блоки и тали". Это может быть нужно, если вы хотите чтобы одна из сторон тали удлинялась быстрее чем другая. Иногда ограничивающая сила на одной стороне слабее чем на другой. Вы можете воспользоваться этим и реализовать механические рычаги.

```
length1 + ratio * length2 == constant
```

Например, если коэффициент ratio равен 2, то lenght1 будет меняться пропорционально удвоенной lenght2. Также сила на отрезке присоединенном к body1 будет равна

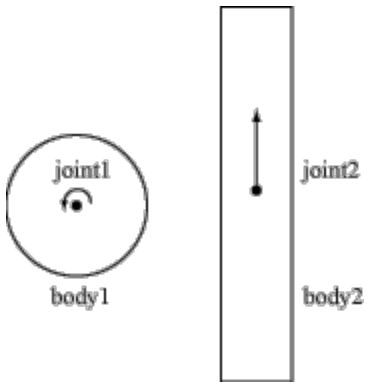
половине сдерживающей силы на отрезке присоединенном к body2.
Когда одна сторона полностью удлинилась, могут возникнуть проблемы. Отрезок на другой стороне получится нулевым. В этом случае уравнение ограничения будет иметь единственное решение (причем плохое). Поэтому талевое соединение ограничивает максимальную длину, которую любой из отрезков может достичь. Хотя, может потребоваться контролировать максимальную длину по геймплейным причинам. Задание максимально-возможной длины для отрезков повышает стабильность и дает больше контроля над ситуацией.

Вот пример определения талевого соединения:

```
 b2Vec2 anchor1 = myBody1->GetWorldCenter();
 b2Vec2 anchor2 = myBody2->GetWorldCenter();
 b2Vec2 groundAnchor1(p1.x, p1.y + 10.0f);
 b2Vec2 groundAnchor2(p2.x, p2.y + 12.0f);
 float32 ratio = 1.0f;
 b2PulleyJointDef jointDef;
 jointDef.Initialize(myBody1, myBody2, groundAnchor1, groundAnchor2,
 anchor1, anchor2, ratio);
 jointDef.maxLength1 = 18.0f;
jointDef.maxLength2 = 20.0f;
```

7.2.5. Передаточное соединение

При создании сложных, хитроумных механизмов не обойтись без использования зубчатых соединений. В принципе, можно создать шестерни, объединяя несколько фигур для моделирования зубьев, но это очень не эффективно и довольно утомительно. Также придется настроить шестерни, чтобы зубья находились в зацеплении. Поэтому Box2D имеет более простой способ: *передаточное соединение*.



Для передаточного соединения необходимо чтобы два тела имели болтовое или призматическое соединение с статичным телом (или с землёй).
Подобно коэффициенту ratio в талевом соединении, передаточное соединение имеет коэффициент передачи. Однако в данном случае коэффициент передачи может быть отрицательным. Также следует помнить, что, если одно соединение болтовое (вращение), а другое призматическое (перемещение), то коэффициент передачи будет иметь размерность длины или единицы делённой на размерность длины.

```
coordinate1 + ratio * coordinate2 == constant
```

Пример использования передаточного соединения:

```
 b2GearJointDef jointDef;
 jointDef.body1 = myBody1;
 jointDef.body2 = myBody2;
 jointDef.joint1 = myRevoluteJoint;
 jointDef.joint2 = myPrismaticJoint;
jointDef.ratio = 2.0f * b2_pi / myLength;
```

Важно помнить, что передаточное соединение зависит от двух других соединений. Что может привести к опасной ситуации. Что произойдет, если эти соединения будут удалены?

> ***Внимание***
> Необходимо сначала удалять передаточное соединение перед удалением болтовых/призматических. Иначе возникнет исключительная ситуация из-за неправильных указателей в передаточном соединении. Также необходимо удалять передаточное соединение перед удалением соединенных тел.

7.2.6. Соединение с курсором мыши

Соединение с курсором мыши используется в тестовом стенде для перетаскивания объектов при помощи мыши. Исходный код находится в b`2MouseJoint.h`.

7.3. Фабрика соединений

Соединения создаются и уничтожаются при помощи методов-фабрик мира. Поэтому следует вспомнить старое замечание:

> ***Внимание***
> Не пытайтесь создавать тела или соединения на стеке или на хипе, используя `new` или `malloc`. Необходимо создавать и удалять тела и соединения только с помощью методов создания и уничтожения тел и соединений класса `b2World`.

Вот пример работы с болтовым (revolute) соединением, от его создания и до уничтожения:

```
 b2RevoluteJointDef jointDef;
 jointDef.body1 = myBody1;
 jointDef.body2 = myBody2;
 jointDef.anchorPoint = myBody1->GetCenterPosition();
 b2RevoluteJoint* joint = myWorld->CreateJoint(&jointDef);
 ... сделать что-нибудь ...
 myWorld->DestroyJoint(joint);
joint = NULL;
```

Считается хорошим тоном обнулять указатели после их уничтожения. Это станет

причиной гарантированного падения программы в случае использования указателя после его уничтожения.
Время жизни соединения не всегда определяется просто. Запомните это предупреждение:

> ***Внимание***
> После уничтожения тела, сразу уничтожаются соединения прикрепленные к нему.

Такая предосторожность не является строгим правилом. Можно так организовать свой игровой движок, что соединения всегда будут уничтожаться до уничтожения присоединенных к ним тел. В этом случае вам не нужно будет реализовывать класс "обработчик" (listener class). Более точную информацию смотрите в Разделе 9.2, "Неявное уничтожение".

7.4. Использование соединений

Часто, соединения создаются и не трогаются до их уничтожения. Однако в соединениях хранится много полезной информации, которую можно использовать для создания более разнообразного моделирования.
Во-первых, можно получить тела, которые соединены, якорную точку и пользовательские данные.

```
 b2Body* GetBody1();
 b2Body* GetBody2();
 b2Vec2 GetAnchor1();
 b2Vec2 GetAnchor2();
void* GetUserData();
```

Все соединения имеют силу и вращающий момент реакции. Это те силы, которые применяются ко второму телу в якорной точке. Эти силы можно использовать для разрыва соединения или для переключения другого игрового события. Эти функции могут выполнять различные вычисления, поэтому их не стоит вызывать слишком часто.

```
 b2Vec2 GetReactionForce();
float32 GetReactionTorque();
```

7.4.1. Использование соединений "жесткий отрезок"

Данные соединения не имеют мускулов или ограничений, поэтому у них нет специфических методов.

7.4.2. Использование болтовых соединений

Можно узнать угол поворота болтового соединения, скорость и вращающий момент мускула.

```
 float32 GetJointAngle() const;
```

```
float32 GetJointSpeed() const;
float32 GetMotorTorque() const;
```

Также можно изменять параметры мускула хоть на каждом шаге.

```
void SetMotorSpeed(float32 speed);
void SetMaxMotorTorque(float32 torque);
```

Мускул соединения имеет интересные возможности. Можно обновлять скорость соединения каждый временной шаг, что позволяет заставить соединение выполнять возвратно поступательные движения или по другому закону в соответствии с вашей функцией.

```
... Начало игрового цикла ...
myJoint->SetMotorSpeed(cosf(0.5f * time));
... Конец игрового цикла ...
```

```
Можно использовать мускул соединения для поддержания определенного угла
соединения. Например:
... Начало игрового цикла ...
float32 angleError = myJoint->GetJointAngle() - angleTarget;
float32 gain = 0.1f;
myJoint->SetMotorSpeed(-gain * angleError);
... Конец игрового цикла ...
```

Обычно параметр усиления (gain) не задаётся слишком большим, иначе соединение может стать нестабильным.

7.4.3. Использование призматических соединений

Использование призматических соединений подобно использованию болтовых. Ниже приведены основные методы соединения:

```
float32 GetJointTranslation() const;
float32 GetJointSpeed() const;
float32 GetMotorForce() const;
void SetMotorSpeed(float32 speed);
void SetMotorForce(float32 force);
```

7.4.4. Использование талевых соединений

Талевые соединения предоставляют нам текущие длины.

```
float32 GetLength1() const;
float32 GetLength2() const;
```

7.4.5. Использование передаточных соединений

Передаточные соединения не предоставляют никакой информации, кроме той, что определена в классе b2Joint.

7.4.6. Использование соединений с курсором мыши

Соединение с курсором мыши позволяет перетаскивать присоединенные объекты, обновляя позицию на каждом шаге.

Глава 8. Контакты

Содержание

8.1. О контактах 8.2. Обработчик контактов 8.3. Фильтрация контактов

8.1. О контактах

Контакты это объекты, создаваемые Box2D для управления столкновениями фигур. Существуют различные виды контактов, которые наследуются от класса b2Contact, для управления контактами между различными типами фигур. Например, существует класс контактов для столкновения полигон-полигон и класс для столкновения окружность-окружность. Это знать не обязательно, можно лишь иметь представление. Существует некоторая терминология, используемая для описания контактов. Эта терминология в основном предназначена для Box2D, но вы можете встретить подобную в других физических движках.

точка контакта (contact point) точка контакта это точка столкновения двух фигур. В реальности объекты имеют пятно контакта, при столкновении поверхностей. Box2D упрощает это, используя лишь небольшое число точек контакта.

нормаль контакта (contact normal)
нормаль контакта это единичный вектор, направленный от shape1 к shape2.

разделение контакта (contact separation)
разделение контакта это противоположность глубины проникновения. Разделение отрицательно, когда фигуры перекрываются. Возможно, в будущих версиях, Box2D сможет создавать точки контакта с положительным разделением и можно будет проверить знак разделения при уведомлении о контактах.

нормальная сила (normal force) Box2D использует итеративный *решальщик* столкновений и хранит результаты вычислений в точках контакта. Можно безболезненно использовать нормальную силу для определения силы удара. Например, можно использовать её для создания разрушающихся предметов или для проигрывания звука при ударе.

касательная сила (tangent force) касательная сила используется для моделирования силы трения.

идентификаторы контактов (contact ids) Box2D, по возможности, использует, вычисленные на предыдущем шаге силы, для контактов следующего шага в качестве начальных условий. Для различения контактов на протяжении шагов моделирования используются идентификаторы контактов. Идентификаторы содержат индексы геометрических параметров, используемые для различения одной точки контакта от другой.

Контакты создаются при пересечении AABB двух фигур. Иногда фильтрация столкновений предотвращает создание контактов. Для Box2d иногда требуется создать контакт, несмотря на фильтрацию столкновений. В таком случае используется b2NullContact, который не подвержен фильтрации. Контакты уничтожаются, когда AABB фигур перестают пересекаться.
Удивительно, но контакты могут создаваться для фигур, которые не пересекаются (пересекаются только их AABB). Это не является неправильной работой, а называется проблемой "курица или яйцо" (что появилось раньше?). Нам неизвестно необходим ли нам объект контакта, пока он не будет создан для анализа столкновения. Мы можем удалить контакт сразу после того как фигуры перестанут пересекаться или просто можем подождать пока перестанут пересекаться AABB фигур. Box2D использует последний подход.

8.2. Обработчик контактов

Можно получать информацию о контактах, реализовав методы класса b2ContactListener (обработчик контактов). При появлении, существовании на протяжении нескольких итераций или уничтожении контакта будет вызываться соответствующий метод обработчика контактов. Следует помнить, что две фигуры могут иметь несколько точек контакта.

```
class MyContactListener : public b2ContactListener
{
	public:
	void Add(const b2ContactPoint* point)
	{
		// обрабатываем добавление контакта
	}

	void Persist(const b2ContactPoint* point)
	{
		// контакт уже существует на протяжении нескольких итераций
	}

	void Remove(const b2ContactPoint* point)
	{
		// обрабатываем удаление контакта
	}

	void Result(const b2ContactResult* point)
	{
		// обрабатываем контакты после вычисления импульсов
	}
	};
```

Внимание
Не следует хранить ссылку на контакт, передаваемый в b2ContactListener. Вместо этого необходимо делать глубокое копирование данных контакта в свой собственный буфер. Пример ниже показывает один из способов сделать это.

Разрешение контактов происходить за несколько итераций, поэтому контакт может быть

добавлен или удалён в течении одного шага моделирования. Поэтому ваш код должен уметь обрабатывать такую ситуацию.
О контактах сообщается сразу же как только они создаются, существуют или удаляются. Это происходит перед тем как будет вызван "решальщик" столкновений, поэтому объект b2ContactPoint не содержит вычисленного импульса. Однако, содержит относительную скорость тел в точке контакта, поэтому можно приблизительно оценить импульс. Если вы реализуете метод Result, то будете получать объект b2ContactResult для точек контакта после вызова "решальщика" столкновений. Этот объект будет содержать импульсы, вычисленные для текущей итерации. Аналогично, метод Result может вызваться несколько раз на протяжении одного шага моделирования.

Кажется заманчивым, реализовать игровую логику, которая изменяет игровой мир при обработке контактов. Например, можно получить столкновение, которое применяет повреждение и пытается уничтожить соответствующий игровой объект и физическое тело. Однако Box2D не позволяет изменять физический мир при обработке контактов, так как можно уничтожить объекты, которые Box2D обрабатывает в данный момент, что приведёт к обращению к недействительным указателям.

Правильным методом обработки контактов считается сохранение нужных контактов в буфер и работа с ними уже после шага моделирования. Необходимо обрабатывать контакты сразу же после шага моделирования, иначе другой код может изменить физический мир, что приведет к недостоверным данным в буфере контактов. При обработке буфера контактов можно изменять физический мир, но это следует делать осторожно, чтобы указатели, хранящиеся в буфере контактов, не стали недействительными. Тестовый стенд содержит пример обработки контактов с защитой от недействительных указателей.

Данный код из теста CollisionProcessing демонстрирует то, как надо работать с удаленными телами при обработке буфера контактов. Желательно внимательно читать комментарии. В этом коде предполагается, что все точки контактов находятся в буфере m_points.

```
// Мы собираемся уничтожить несколько тел в зависимости от контактов.
// Нам необходимо сохранить тела которые необходимо уничтожить,
// так как между ними может содержаться несколько контактов.

const int32 k_maxNuke = 6;
b2Body* nuke[k_maxNuke];
int32 nukeCount = 0;

// Пробегаемся по буферу контактов. Уничтожаем тела которые
// столкнулись с более тяжелыми телами

for (int32 i = 0; i < m_pointCount; ++i)
{
     ContactPoint* point = m_points + i;

     b2Body* body1 = point->shape1->GetBody();
     b2Body* body2 = point->shape2->GetBody();
     float32 mass1 = body1->GetMass();
     float32 mass2 = body2->GetMass();

     if (mass1 > 0.0f && mass2 > 0.0f)
     {
```

```
                if (mass2 > mass1)
            {
                nuke[nukeCount++] = body1;
            }
            else
            {
              nuke[nukeCount++] = body2;
            }

            if (nukeCount == k_maxNuke)
            {
                break;
            }
        }
    }

    // Сортируем массив nuke для того чтобы сгруппировать одни и те же тела
    std::sort(nuke, nuke + nukeCount);

    // Уничтожаем тела, пропуская дубликаты
    int32 i = 0;
    while (i < nukeCount)
    {
        b2Body* b = nuke[i++];
        while (i < nukeCount && nuke[i] == b)
        {
            ++i;
        }

        m_world->DestroyBody(b);
}
```

8.3. Фильтрация контактов

Часто в игре требуется, чтобы некоторые тела не сталкивались. Например, может потребоваться создать дверь, через которую могут проходить только определённые персонажи. Это называется фильтрацией контактов, так как некоторые столкновения отфильтровываются.
Box2D позволяет добиться строго определённой фильтрации контактов, для этого необходимо реализовать класс b2ContactFilter. В этом классе необходимо реализовать метод ShouldCollide, который принимает два указателя на фигуры. Этот метод возвращает истину, если фигуры должны взаимодействовать.
Стандартная реализация метода ShouldCollide, использующая b2FilterData, определена в Глава 6, Фигуры.

```
bool b2ContactFilter::ShouldCollide(
    b2Shape* shape1, b2Shape* shape2
)
{
    const b2FilterData& filter1 = shape1->GetFilterData();
```

```
	const b2FilterData& filter2 = shape2->GetFilterData();

	if (
		filter1.groupIndex == filter2.groupIndex &&
		filter1.groupIndex != 0
	)
	{
		 return filter1.groupIndex > 0;
	}

	 bool collide =
		(filter1.maskBits & filter2.categoryBits) != 0 &&
		filter1.categoryBits & filter2.maskBits) != 0;

	return collide;
}
```

Глава 9. Напоследок

Содержание

9.1. Границы мира 9.2. Неявное уничтожение

9.1. Границы мира

Можно реализовать b2BoundaryListener, который позволит миру информировать вас о телах, вылетевших за пределы ограничивающего бокса мира. При получении уведомления нельзя пытаться удалять тело, вместо этого необходимо пометить игровой объект для удаления или ошибки. И удалить тело после шага моделирования физики.

```
 class MyBoundaryListener : public b2BoundaryListener
 {
	 void Violation(b2Body* body)
	 {
		 MyActor* myActor = (MyActor*)body->GetUserData();
		 myActor->MarkForErrorHandling();
	 }
};
```

Затем необходимо подключить данный обработчик к объекту мира. Это необходимо сделать при инициализации мира.

```
myWorld->SetListener(myBoundaryListener);
```

9.2. Неявное уничтожение

Box2D не использует подсчёт ссылок. Поэтому если вы уничтожаете тело, то оно действительно уничтожается. Попытка доступа к указателю на уничтоженное тело

может иметь неопределённое поведение. Для решения таких проблем, отладочная версия менеджера памяти заполняет уничтоженные объекты значениями FDFDFDFD. Это помогает быстрее найти проблемы в некоторых случаях.
При удалении объекта Box2D, в первую очередь необходимо убедиться, что также были удалены или обнулены все ссылки на удалённый объект. Это просто если имеется одна ссылка на объект. Если имеется несколько ссылок, то можно задуматься над реализацией обёртки над ссылкой на объект Box2D.
Часто, при использовании Box2D, будет создаваться множество тел, фигур и соединений. Работа с этими объектами частично автоматизирована в Box2D. Если удаляется тело, то и все связанные с ним фигуры и соединения удаляются автоматически. Это называется - *неявное уничтожение*. При этом освобождается/будится тело находящееся на другом конце соединения или соприкасающееся.

Этот процесс обычно удобен, но следует помнить о критическом моменте

> ***Внимание***
> При удалении тела, все фигуры и соединения связанные с телом автоматически удаляются. Необходимо обнулять все имеющиеся указатели на эти фигуры и соединения. Иначе, позднее ваша программа рухнет при попытке доступа или уничтожения этих фигур или соединений.

Для помощи вам в обнулении указателей на соединения, Box2D предоставляет класс b2WorldListener, который вы можете реализовать и зарегистрировать в объекте мира. После этого, объект мира будет уведомлять вас перед неявным удалением соединения. Неявное уничтожение в большинстве случаев экономит много времени. Но также может привести к падению программы. Вы можете хранить указатели на фигуры и соединения где-то в вашем коде. Эти указатели становятся недействительными, когда удаляется соответствующее тело. Ситуация ухудшается, когда соединения создаются частью кода не имеющего отношения к обработке соответствующего тела. Например, тестовый стенд создает b2MouseJoint для манипуляций над телами на экране.
Box2D предоставляет механизм оповещения о попытке неявного уничтожения. Это позволяет вашему приложению обнулить все недействительные указатели. Для этого необходимо реализовать класс b2DestructionListener, который позволяет миру информировать вас о неявном уничтожении фигуры или соединения, если уничтожено соответствующее тело. Это помогает предотвратить доступ к недействительным указателям.

```
class MyDestructionListener : public b2DestructionListener
{
        void SayGoodbye(b2Joint* joint)
        {
                // удаление всех ссылок на соединение
        }
};
```

Класс обработчика неявных уничтожений нужно зарегистрировать в объекте мира. Это надо сделать при инициализации мира.

```
myWorld->SetListener(myDestructionListener);
```

Глава 10. Настройки

Содержание

10.1. О настройках 10.2. Отклонения 10.3. Выделение памяти

10.1. О настройках

Имеется два исходных файла содержащиеся в Box2D специально для настроек пользователя. Это b2Settings.h и b2Settings.cpp.
Box2D использует числа с плавающей точкой, поэтому необходимо использовать некоторые отклонения для правильной работы Box2D.

10.2. Отклонения

Имеется множество отклонений зависящих от используемых единиц измерения. В подразделе 3.3, Единицы измерения более подробно описана используемая система единиц измерения.
В doxygen docs вы найдёте описание каждого отклонения в отдельности.

10.3. Выделение памяти

Вся работа Box2D с памятью осуществляется через b2Alloc и b2Free (файл b2Settings.cpp) за исключением следующих случаев:

- Объект b2World может быть создан как локальная переменная или любым другим способом по вашему усмотрению.
- Любой другой класс Box2D создаваемый без использования фабричного метода. Это включает такие объекты как обработчики и буферы контактов.

Способ выделения памяти может быть изменён в b2Settings.cpp.

Глава 11. Отладочная отрисовка

Вы можете реализовать класс b2DebugDraw (файл b2WorldCallbacks.h) чтобы получить возможность отрисовать физические сущности во всех деталях:

- контуры фигур
- соединения
- core shapes (для continuous collision)
- ограничивающие боксы (AABBs), включая AABB мира

- ориентированные боксы (OBBs)
- пары потенциальных столкновений (потенциальные контакты)
- центры массы

Такой метод отрисовки физических объектов предпочтительнее, чем прямой доступ. Причина этого в том, что нужная информация является внутренней и постоянно меняется.

Тестовый стенд отрисовывает физические объекты используя отладочную отрисовку и обработчик контактов и служит прекрасным примером того, как производить отладочную отрисовку и отрисовку точек контакта.